

Exp4

SA25011052 刘睿博

Rotate

```
void rotate(int dim, pixel *src, pixel *dst)
{
    int stride = 8;
    for(int i = 0; i < dim; i+=stride) {
        for(int j = 0; j < dim; j+= stride) {
            for(int k = i; k < i+stride; k++) {
                dst[RIDX(dim-1-j, k, dim)] = src[RIDX(k, j+0, dim)];
                dst[RIDX(dim-2-j, k, dim)] = src[RIDX(k, j+1, dim)];
                dst[RIDX(dim-3-j, k, dim)] = src[RIDX(k, j+2, dim)];
                dst[RIDX(dim-4-j, k, dim)] = src[RIDX(k, j+3, dim)];
                dst[RIDX(dim-5-j, k, dim)] = src[RIDX(k, j+4, dim)];
                dst[RIDX(dim-6-j, k, dim)] = src[RIDX(k, j+5, dim)];
                dst[RIDX(dim-7-j, k, dim)] = src[RIDX(k, j+6, dim)];
                dst[RIDX(dim-8-j, k, dim)] = src[RIDX(k, j+7, dim)];
            }
        }
    }
}
```

使用循环展开，对8*8的小矩阵进行操作

Smooth

```
void smooth(int dim, pixel *src, pixel *dst)
{
    for(int i = 1; i < dim; i++) {
        // top edge
        dst[RIDX(i, 0, dim)].blue = (src[RIDX(i-1, 0, dim)].blue + src[RIDX(i-1, 1,
dim)].blue +
                                     src[RIDX(i, 0, dim)].blue + src[RIDX(i, 1, dim)].blue +
                                     src[RIDX(i+1, 0, dim)].blue + src[RIDX(i+1, 1, dim)].blue) /
6;
        dst[RIDX(i, 0, dim)].green = (src[RIDX(i-1, 0, dim)].green + src[RIDX(i-1, 1,
dim)].green +
                                     src[RIDX(i, 0, dim)].green + src[RIDX(i, 1, dim)].green +
                                     src[RIDX(i+1, 0, dim)].green + src[RIDX(i+1, 1, dim)].green)
/ 6;
        dst[RIDX(i, 0, dim)].red = (src[RIDX(i-1, 0, dim)].red + src[RIDX(i-1, 1, dim)].red +
                                     src[RIDX(i, 0, dim)].red + src[RIDX(i, 1, dim)].red +
                                     src[RIDX(i+1, 0, dim)].red + src[RIDX(i+1, 1, dim)].red)
/ 6;
    }
}
```

```

src[RIDX(i+1, 0, dim)].red + src[RIDX(i+1, 1, dim)].red) / 6;
// bottom edge
dst[RIDX(i, dim-1, dim)].blue = (src[RIDX(i-1, dim-2, dim)].blue + src[RIDX(i-1,
dim-1, dim)].blue +
src[RIDX(i, dim-2, dim)].blue + src[RIDX(i, dim-1, dim)].blue
+
src[RIDX(i+1, dim-2, dim)].blue + src[RIDX(i+1, dim-1,
dim)].blue) / 6;
dst[RIDX(i, dim-1, dim)].green = (src[RIDX(i-1, dim-2, dim)].green + src[RIDX(i-1,
dim-1, dim)].green +
src[RIDX(i, dim-2, dim)].green + src[RIDX(i, dim-1,
dim)].green +
src[RIDX(i+1, dim-2, dim)].green + src[RIDX(i+1, dim-1,
dim)].green) / 6;
dst[RIDX(i, dim-1, dim)].red = (src[RIDX(i-1, dim-2, dim)].red + src[RIDX(i-1, dim-1,
dim)].red +
src[RIDX(i, dim-2, dim)].red + src[RIDX(i, dim-1, dim)].red +
src[RIDX(i+1, dim-2, dim)].red + src[RIDX(i+1, dim-1,
dim)].red) / 6;

// left edge
dst[RIDX(0, i, dim)].blue = (src[RIDX(0, i-1, dim)].blue + src[RIDX(0, i, dim)].blue
+
src[RIDX(1, i-1, dim)].blue + src[RIDX(1, i, dim)].blue +
src[RIDX(0, i+1, dim)].blue + src[RIDX(1, i+1, dim)].blue) /
6;
dst[RIDX(0, i, dim)].green = (src[RIDX(0, i-1, dim)].green + src[RIDX(0, i,
dim)].green +
src[RIDX(1, i-1, dim)].green + src[RIDX(1, i, dim)].green +
src[RIDX(0, i+1, dim)].green + src[RIDX(1, i+1, dim)].green)
/ 6;
dst[RIDX(0, i, dim)].red = (src[RIDX(0, i-1, dim)].red + src[RIDX(0, i, dim)].red +
src[RIDX(1, i-1, dim)].red + src[RIDX(1, i, dim)].red +
src[RIDX(0, i+1, dim)].red + src[RIDX(1, i+1, dim)].red) / 6;

// right edge
dst[RIDX(dim-1, i, dim)].blue = (src[RIDX(dim-2, i-1, dim)].blue + src[RIDX(dim-2, i,
dim)].blue +
src[RIDX(dim-1, i-1, dim)].blue + src[RIDX(dim-1, i,
dim)].blue +
src[RIDX(dim-2, i+1, dim)].blue + src[RIDX(dim-1, i+1,
dim)].blue) / 6;
dst[RIDX(dim-1, i, dim)].green = (src[RIDX(dim-2, i-1, dim)].green + src[RIDX(dim-2,
i, dim)].green +
src[RIDX(dim-1, i-1, dim)].green + src[RIDX(dim-1, i,
dim)].green +
src[RIDX(dim-2, i+1, dim)].green + src[RIDX(dim-1, i+1,
dim)].green) / 6;
dst[RIDX(dim-1, i, dim)].red = (src[RIDX(dim-2, i-1, dim)].red + src[RIDX(dim-2, i,
dim)].red +

```

```

src[RIDX(dim-1, i-1, dim)].red + src[RIDX(dim-1, i, dim)].red
+
src[RIDX(dim-2, i+1, dim)].red + src[RIDX(dim-1, i+1,
dim)].red) / 6;
}
// four corners
dst[RIDX(0, 0, dim)].blue = (src[RIDX(0, 0, dim)].blue + src[RIDX(0, 1, dim)].blue +
src[RIDX(1, 0, dim)].blue + src[RIDX(1, 1, dim)].blue) / 4;
dst[RIDX(0, 0, dim)].green = (src[RIDX(0, 0, dim)].green + src[RIDX(0, 1, dim)].green +
src[RIDX(1, 0, dim)].green + src[RIDX(1, 1, dim)].green) / 4;
dst[RIDX(0, 0, dim)].red = (src[RIDX(0, 0, dim)].red + src[RIDX(0, 1, dim)].red +
src[RIDX(1, 0, dim)].red + src[RIDX(1, 1, dim)].red) / 4;
dst[RIDX(0, dim-1, dim)].blue = (src[RIDX(0, dim-2, dim)].blue + src[RIDX(0, dim-1,
dim)].blue +
src[RIDX(1, dim-2, dim)].blue + src[RIDX(1, dim-1, dim)].blue) /
4;
dst[RIDX(0, dim-1, dim)].green = (src[RIDX(0, dim-2, dim)].green + src[RIDX(0, dim-1,
dim)].green +
src[RIDX(1, dim-2, dim)].green + src[RIDX(1, dim-1, dim)].green)
/ 4;
dst[RIDX(0, dim-1, dim)].red = (src[RIDX(0, dim-2, dim)].red + src[RIDX(0, dim-1,
dim)].red +
src[RIDX(1, dim-2, dim)].red + src[RIDX(1, dim-1, dim)].red) / 4;
dst[RIDX(dim-1, 0, dim)].blue = (src[RIDX(dim-2, 0, dim)].blue + src[RIDX(dim-2, 1,
dim)].blue +
src[RIDX(dim-1, 0, dim)].blue + src[RIDX(dim-1, 1, dim)].blue) /
4;
dst[RIDX(dim-1, 0, dim)].green = (src[RIDX(dim-2, 0, dim)].green + src[RIDX(dim-2, 1,
dim)].green +
src[RIDX(dim-1, 0, dim)].green + src[RIDX(dim-1, 1, dim)].green)
/ 4;
dst[RIDX(dim-1, 0, dim)].red = (src[RIDX(dim-2, 0, dim)].red + src[RIDX(dim-2, 1,
dim)].red +
src[RIDX(dim-1, 0, dim)].red + src[RIDX(dim-1, 1, dim)].red) / 4;
dst[RIDX(dim-1, dim-1, dim)].blue = (src[RIDX(dim-2, dim-2, dim)].blue + src[RIDX(dim-2,
dim-1, dim)].blue +
src[RIDX(dim-1, dim-2, dim)].blue + src[RIDX(dim-1, dim-1,
dim)].blue) / 4;
dst[RIDX(dim-1, dim-1, dim)].green = (src[RIDX(dim-2, dim-2, dim)].green +
src[RIDX(dim-2, dim-1, dim)].green +
src[RIDX(dim-1, dim-2, dim)].green + src[RIDX(dim-1, dim-1,
dim)].green) / 4;
dst[RIDX(dim-1, dim-1, dim)].red = (src[RIDX(dim-2, dim-2, dim)].red + src[RIDX(dim-2,
dim-1, dim)].red +
src[RIDX(dim-1, dim-2, dim)].red + src[RIDX(dim-1, dim-1,
dim)].red) / 4;
// center area
for(int i = 1; i < dim-1; i++) {
    for(int j = 1; j < dim-1; j++) {

```

```

        dst[RIDX(i, j, dim)].blue = (src[RIDX(i-1, j-1, dim)].blue + src[RIDX(i-1, j,
dim)].blue + src[RIDX(i-1, j+1, dim)].blue +
                                src[RIDX(i, j-1, dim)].blue + src[RIDX(i, j, dim)].blue +
src[RIDX(i, j+1, dim)].blue +
                                src[RIDX(i+1, j-1, dim)].blue + src[RIDX(i+1, j,
dim)].blue + src[RIDX(i+1, j+1, dim)].blue) / 9;
        dst[RIDX(i, j, dim)].green = (src[RIDX(i-1, j-1, dim)].green + src[RIDX(i-1, j,
dim)].green + src[RIDX(i-1, j+1, dim)].green +
                                src[RIDX(i, j-1, dim)].green + src[RIDX(i, j, dim)].green
+ src[RIDX(i, j+1, dim)].green +
                                src[RIDX(i+1, j-1, dim)].green + src[RIDX(i+1, j,
dim)].green + src[RIDX(i+1, j+1, dim)].green) / 9;
        dst[RIDX(i, j, dim)].red = (src[RIDX(i-1, j-1, dim)].red + src[RIDX(i-1, j,
dim)].red + src[RIDX(i-1, j+1, dim)].red +
                                src[RIDX(i, j-1, dim)].red + src[RIDX(i, j, dim)].red +
src[RIDX(i, j+1, dim)].red +
                                src[RIDX(i+1, j-1, dim)].red + src[RIDX(i+1, j, dim)].red
+ src[RIDX(i+1, j+1, dim)].red) / 9;
    }
}
}

```

对边角进行平滑处理后再处理中间部分，可以省去边界判断

```
● (base) jyjs@VServer:~/Documents/CSAPP/Exp/Exp4/perflab-handout$ ./driver
Teamname: liuruibo
Member 1: liuruibo
Email 1: liuruibo21@mail.ustc.edu.cn
```

```
Rotate: Version = naive_rotate: Naive baseline implementation:
Dim          256    512    1024    2048    4096    Mean
Your CPEs    3.6    4.6    6.5    8.5    25.0
Baseline CPEs 14.7   40.1   46.4   65.9   94.5
Speedup      4.1    8.7    7.1    7.8    3.8    5.9
```

```
Rotate: Version = rotate: Current working version:
Dim          256    512    1024    2048    4096    Mean
Your CPEs    1.2    1.6    1.6    2.5    4.6
Baseline CPEs 14.7   40.1   46.4   65.9   94.5
Speedup      12.6   25.6   28.4   25.9   20.5   21.8
```

```
Smooth: Version = smooth: Current working version:
Dim          256    512    1024    2048    4096    Mean
Your CPEs    7.0    6.7    6.8    6.9    7.0
Baseline CPEs 695.0   698.0   702.0   717.0   722.0
Speedup      100.0   104.8   102.7   103.5   102.5   102.7
```

```
Smooth: Version = naive_smooth: Naive baseline implementation:
Dim          256    512    1024    2048    4096    Mean
Your CPEs    17.4   17.6   17.7   17.8   18.2
Baseline CPEs 695.0   698.0   702.0   717.0   722.0
Speedup      40.0   39.8   39.6   40.2   39.7   39.9
```

Summary of Your Best Scores:

Rotate: 21.8 (rotate: Current working version)

Smooth: 102.7 (smooth: Current working version)